

Python and ML Programming II

Placement-oriented notes on practical ML engineering. Code is intentionally small but follows production habits: separate validation data, version artifacts, and validate inputs.

Pipelines

1. Overview

A pipeline is an ordered, reproducible chain of preprocessing and model steps. It prevents training-serving mismatch in tabular scoring systems.

2. Intuition

It is a factory line: raw rows always receive the same cleaning, encoding, and prediction treatment.

3. Prerequisites

Python, Pandas, train/test splits, estimators, and data leakage.

4. Core Concepts

Subtopic	Meaning / why it matters	Interview angle
Transformer	Learns state with fit , applies it with transform ; e.g., scaler.	Why fit only on train?
Estimator	Final fit/predict model.	Why is it last?
ColumnTransformer	Different rules for numeric and categorical columns.	How handle mixed schema?
CV	Repeats train/validation folds safely.	How does pipeline prevent leakage?

5. Algorithm / Working Process

Split raw data; fit transforms on training rows; transform train/validation; fit estimator; persist the entire object; call it on raw serving input.

6. Mathematical Foundation

Standardization is $z = (x - \mu_{train}) / \sigma_{train}$. The train-only subscript is crucial: test statistics leak information.

7. Practical Implementation

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

prep = ColumnTransformer([
    ("num", Pipeline([("fill", SimpleImputer(strategy="median")), ("scale",
StandardScaler())])), [ "age", "income" ]),
    ("cat", Pipeline([("fill", SimpleImputer(strategy="most_frequent")), ("oh",
OneHotEncoder(handle_unknown="ignore"))])), [ "city" ]),
])
pipe = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=1000))])
pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

```

8. Code Explanation

Nested pipelines impute before transformation. Unknown categories are safe at serving time. The outer object guarantees preprocessing and classifier stay version-aligned.

9. Training / Evaluation

Use stratification for class imbalance and chronological splits for time data. Evaluate held-out data with F1/PR-AUC when false negatives/positives matter; tune `model__C` with CV.

10. Complexity and Cost

Scaling is $O(nd)$; one-hot features can create large sparse matrices. Cost depends mostly on the final estimator.

11. Common Use Cases

Churn, credit risk, demand forecasting features, and repeatable batch scoring.

12. Common Mistakes

Fitting before splitting, target leakage, random time splits, a different serving transform, and no unknown-category policy.

13. Edge Cases / Limitations

Pipelines do not detect drift or invalid schemas; validate schema and version the feature definition separately.

14. Variations

`GridSearchCV` can wrap a pipeline; `FeatureUnion` combines branches; Airflow/Kubeflow orchestrate jobs rather than in-memory transforms.

15. Related Topics

Feature engineering creates steps; MLflow tracks artifacts; FastAPI serves the serialized pipeline.

16. Interview Questions

1. Why pipelines? Reproducibility and leakage prevention.
2. `fit_transform` vs `transform`? Learn-and-apply versus apply-only.
3. Why train-only scaler? Test statistics are unavailable in production.
4. Why `ColumnTransformer`? Columns have different data types.
5. How tune it? `step__parameter` names in CV.
6. Can it select features? Yes, as an intermediate transformer.
7. Why one artifact? Prevent model/preprocessor mismatch.
8. How handle unseen categories? Ignore, unknown bucket, or robust encoding.
9. Why time split? To simulate future prediction.
10. Pipeline vs DAG? Estimator composition versus workflow scheduling.

17. Practice Tasks

Build a Titanic pipeline; demonstrate leakage intentionally; add CV; save/reload; test an unseen category.

18. Project Ideas

Project	Stack / dataset	Resume value
Loan scorer	sklearn/FastAPI; lending data	Leakage-safe deployment
Churn baseline	Pandas/sklearn; Telco	Mixed-feature handling
Batch scorer	sklearn/MLflow	Feature-to-artifact story

19. Quick Revision

Key idea: fit transforms on train only. Formula: train-only z-score. Trap: preprocessing before split. One-liner: a pipeline packages transformations and estimator into one safe artifact.

20. Final Cheat Sheet

Input/output: raw table → prediction. **Steps:** split, fit, train, predict. **Metrics:** task-specific. **Pros:** reproducible. **Con:** can hide expensive transforms. **Best:** tabular ML.

PyTorch

1. Overview

PyTorch is a tensor framework with GPU support and automatic differentiation, used for research and production deep learning.

2. Intuition

Tensors are GPU-capable arrays; autograd remembers operations so a loss can tell each weight how to change.

3. Prerequisites

NumPy shapes, calculus derivatives, gradient descent, neural networks, Python OOP, and GPU memory basics.

4. Core Concepts

Concept	Meaning / example	Interview angle
Tensor	Array with dtype/device.	CPU vs CUDA tensor?
Autograd	Dynamic graph differentiation.	Why <code>zero_grad()</code> ?
<code>nn.Module</code>	Registered parameters and forward logic.	Why layers in <code>__init__</code> ?
<code>DataLoader</code>	Batching/shuffling data.	Why pinned memory?
<code>train/eval</code>	Changes dropout and batch norm.	Is it enough to disable gradients?

5. Algorithm / Working Process

Batch input → forward logits → loss → **backward** accumulates gradients → optimizer update → clear gradients. Inference uses `eval()` and `inference_mode()`.

6. Mathematical Foundation

Gradient descent: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$. Backprop uses the chain rule. Cross entropy penalizes low probability on the true class.

7. Practical Implementation

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

X = torch.randn(400, 2); y = (X.sum(1) > 0).long()
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)
device = "cuda" if torch.cuda.is_available() else "cpu"
net = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 2)).to(device)
opt, loss_fn = torch.optim.Adam(net.parameters(), lr=1e-3), nn.CrossEntropyLoss()
for epoch in range(20):
    net.train()
    for xb, yb in loader:
        loss = loss_fn(net(xb.to(device)), yb.to(device))
        opt.zero_grad(); loss.backward(); opt.step()
net.eval()
with torch.inference_mode(): print(net(X[:2].to(device)).argmax(1))
```

8. Code Explanation

Cross entropy expects logits and integer labels. Parameters and batches must use the same device. Gradients accumulate by design, hence `zero_grad` precedes backpropagation.

9. Training / Evaluation

Keep a validation set; report loss plus accuracy/F1/AUC as appropriate. Rising validation loss with falling train loss indicates overfitting. Tune LR first, then batch size, architecture, augmentation, regularization, and schedule.

10. Complexity and Cost

Dense layer cost is $O(Bd_{in}d_{out})$; activations and autograd graph consume memory. AMP can reduce memory and improve GPU throughput.

11. Common Use Cases

CNNs, transformers, diffusion, reinforcement learning, and custom research models.

12. Common Mistakes

Softmax before `CrossEntropyLoss`, absent `zero_grad`, device mismatch, wrong `train/eval`, logging graph-bearing tensors, and evaluating training data.

13. Edge Cases / Limitations

NaNs, OOM, nondeterministic kernels, and broadcasting shape errors require checks and profiling.

14. Variations

`torch.compile`, AMP, DDP, FSDP, and Lightning. Core PyTorch is crucial for placements and research.

15. Related Topics

CUDA executes operations; Triton writes custom kernels; DDP synchronizes gradients; Transformers commonly run on PyTorch.

16. Interview Questions

1. What is autograd? Dynamic graph differentiation.
2. Why clear gradients? They accumulate across `backward` calls.
3. Logits vs probabilities? Raw scores versus normalized values.
4. Why logits in loss? Numerical stability.
5. Why `eval()`? Correct dropout and batch-norm behavior.
6. State dict? Parameter/buffer mapping.
7. Why shuffle? Reduce order bias.
8. OOM fixes? Smaller batch, AMP, checkpointing, smaller model.
9. `no_grad` vs inference mode? Both disable grads; inference mode adds restrictions/optimization.
10. What does backward return? Gradients for leaf parameters.

17. Practice Tasks

Implement linear regression manually; train MNIST; add early stopping; debug a device mismatch; compare FP32 and AMP.

18. Project Ideas

Project	Stack / dataset	Resume value
Defect classifier	PyTorch/torchvision; MVTec subset	Vision loop
Sentiment model	PyTorch; IMDB	Sequence basics
Fraud detector	PyTorch; credit-card data	Imbalance handling

19. Quick Revision

Tensor + autograd + modules. $\theta \leftarrow \theta - \eta \nabla L$. Trap: wrong loss/input format. One-liner: PyTorch records tensor operations and backpropagates gradients.

20. Final Cheat Sheet

Input/output: tensors $\rightarrow$ logits/loss. **Steps:** loader, forward, loss, backward, step. **Keys:** LR/batch/model. **Pros:** flexible. **Con:** explicit loop responsibility. **Best:** custom deep learning.

TensorFlow/Keras

1. Overview

TensorFlow is a numerical/deployment ecosystem; Keras is its high-level model API. It is useful for rapid, standard neural-network development.

2. Intuition

Keras layers are LEGO bricks: `compile` states how to learn and `fit` runs the usual loop.

3. Prerequisites

NumPy, layers, losses, optimizers, and validation methodology.

4. Core Concepts

Concept	Meaning / why	Interview angle
Model	Sequential, Functional, or subclassed graph.	When Functional API?
Compile	Optimizer, loss, metrics declaration.	Loss vs metric?
<code>tf.data</code>	Streaming input pipeline.	Why prefetch?
Callback	Automated checkpointing/stopping.	Why restore best weights?

5. Algorithm / Working Process

Define input-output graph; compile; batch data; Keras forward-propagates, differentiates internally, updates weights, validates, then exports predictions.

6. Mathematical Foundation

For batch B , $L_B = |B|^{-1} \sum_i l(y_i, f_\theta(x_i))$. Softmax is $e^{z_k} / \sum_j e^{z_j}$; an optimizer minimizes cross entropy.

7. Practical Implementation

```
import tensorflow as tf
X = tf.random.normal((400, 2)); y = tf.cast(tf.reduce_sum(X, axis=1) > 0,
tf.int32)
model = tf.keras.Sequential([tf.keras.Input((2,)), tf.keras.layers.Dense(16,
activation="relu"), tf.keras.layers.Dense(2)])
model.compile("adam",
tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
model.fit(X, y, validation_split=.2, epochs=20,
        callbacks=[tf.keras.callbacks.EarlyStopping(patience=3,
restore_best_weights=True)])
print(tf.argmax(model(X[:2], training=False), axis=1))
```

8. Code Explanation

Final layer returns logits, so `from_logits=True` is necessary. Early stopping restores the best validation checkpoint. Explicit validation splits are better for time/grouped data.

9. Training / Evaluation

Use `Dataset.shuffle().batch().prefetch(tf.data.AUTOTUNE)`. Evaluate holdout data; use precision/recall/AUC for imbalance. Save model and preprocessing together.

10. Complexity and Cost

Architecture determines compute. Batch size trades GPU utilization against activation memory; input pipelines can bottleneck training.

11. Common Use Cases

Image classification, mobile inference via TFLite, structured-data networks, and TF.js/web applications.

12. Common Mistakes

Loss-label mismatch, double softmax, leakage, accuracy-only reporting, and training-only augmentation used at inference.

13. Edge Cases / Limitations

Tracing Python side effects can surprise users; complex custom research loops may be clearer in lower-level frameworks.

14. Variations

Sequential for linear stacks; Functional for multi-input/output and residual graphs; subclassing for bespoke control. Functional API is interview-relevant.

15. Related Topics

TFLite/Serving deploy models; PyTorch exposes a more explicit loop; distributed strategies scale training.

16. Interview Questions

1. TensorFlow vs Keras? Ecosystem/runtime versus high-level API.
2. Sequential limitation? Linear single-input/output stack.
3. Why logits? Stable cross entropy.
4. Loss vs metric? Optimized objective versus report.
5. What is `tf.data`? Efficient composable input pipeline.
6. Why prefetch? Overlaps input prep with device compute.
7. Why callbacks? Stop/save/schedule automatically.
8. When custom loop? Unusual update logic.
9. SavedModel? Portable export format.
10. Why `training=False`? Correct dropout/batch norm inference.

17. Practice Tasks

Build MNIST with `tf.data`; make a residual Functional model; add class weights; convert a model to TFLite; write a GradientTape loop.

18. Project Ideas

Project	Stack / dataset	Resume value
Mobile digit recognizer	Keras/TFLite; MNIST	Edge delivery
Multi-input price model	Functional Keras	Graph modeling
Waste sorter	Keras; TrashNet	Applied CV

19. Quick Revision

Define, compile, fit, evaluate. Softmax CE uses logits. Trap: loss mismatch. One-liner: compile declares learning; fit executes it.

20. Final Cheat Sheet

Input/output: tensors → outputs. **Steps:** define/compile/fit/export. **Keys:** optimizer/LR/epochs. **Pros:** concise. **Con:** abstraction can hide custom behavior. **Best:** rapid DL.

Hugging Face Transformers

1. Overview

Transformers provides pretrained models, tokenizers, datasets integration, and inference/training utilities for NLP, vision, audio, and multimodal AI.

2. Intuition

A tokenizer turns text into IDs; attention lets each token look at useful context; pretraining supplies broad knowledge before task adaptation.

3. Prerequisites

PyTorch or TensorFlow, embeddings, softmax/cross entropy, attention, GPU basics, and careful splits.

4. Core Concepts

Concept	Meaning / why	Interview angle
Tokenizer	Text $\leftrightarrow$ IDs and masks.	Why match tokenizer/checkpoint?
Checkpoint	Architecture config plus weights.	What Auto classes infer?
Attention mask	Marks real tokens vs padding.	Why required?
Fine-tuning	Updates pretrained weights.	Full tune vs LoRA?
Pipeline	Simple inference wrapper.	Why not training interface?

5. Algorithm / Working Process

Tokenize with truncation/padding; embed IDs/positions; run attention and feed-forward blocks; task head emits logits/tokens; calculate loss for fine-tuning or decode tokens for generation.

6. Mathematical Foundation

Attention is $\text{softmax}(QK^T/\sqrt{d_k})V$. Causal LM minimizes $-\sum_t \log p(x_t | x_{<t})$. Temperature changes sampling through $\text{softmax}(z/T)$.

7. Practical Implementation

```
from transformers import pipeline, AutoTokenizer,
AutoModelForSequenceClassification
import torch
name = "distilbert-base-uncased-finetuned-sst-2-english"
print(pipeline("sentiment-analysis", model=name)("The API is reliable.))
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(name)
batch = tok(["great model", "poor latency"], padding=True, truncation=True,
```

```
return_tensors="pt")
with torch.inference_mode(): probs = model(**batch).logits.softmax(-1)
print(probs)
```

8. Code Explanation

`pipeline` bundles preprocessing and postprocessing. Explicit calls reveal `input_ids`, masks, and logits—the data needed to debug and train. Always load the tokenizer from the same checkpoint.

9. Training / Evaluation

Tokenize batches; use dynamic padding; preserve document/user grouping in splits. Evaluate classifiers with F1 and generative systems with task metrics plus human/safety evaluation. Tune LR, length, batch, epochs, and adapters.

10. Complexity and Cost

Vanilla attention is $O(n^2d)$. Large models need substantial GPU memory; LoRA and quantization reduce fine-tuning cost. Autoregressive inference grows with generated tokens.

11. Common Use Cases

Classification, NER, translation, summarization, Q&A, embedding retrieval, vision models, and assistants.

12. Common Mistakes

Tokenizer mismatch, destructive truncation, duplicate documents across splits, one-metric evaluation, and treating generation as factually reliable.

13. Edge Cases / Limitations

Hallucination, bias, prompt sensitivity, stale knowledge, context limits, and costly long prompts remain.

14. Variations

Encoder-only BERT for understanding; causal GPT-like decoders for generation; T5-style encoder-decoder models; LoRA/QLoRA for efficient adaptation.

15. Related Topics

PyTorch provides execution; RAG grounds generation; CUDA/Triton optimize operations; MLflow records fine-tunes.

16. Interview Questions

1. Why pretraining? Transfers useful representations.
2. What is tokenization? Mapping text to vocabulary IDs.
3. Why attention mask? Ignore padding.
4. BERT vs GPT? Bidirectional encoder versus causal decoder.

5. Why scale attention? Stable dot-product magnitudes.
6. Fine-tuning? Updating a checkpoint for a task.
7. LoRA? Low-rank trainable adapter updates.
8. Why dynamic padding? Less wasted compute.
9. Temperature? Higher gives more diverse sampling.
10. Reduce hallucination? Retrieval, constraints, verification, evaluation—not guarantee.

17. Practice Tasks

Fine-tune DistilBERT on AG News; inspect tokenization; compare max lengths; add LoRA; analyze errors by class/length.

18. Project Ideas

Project	Stack / dataset	Resume value
Ticket router	Transformers/FastAPI; CLINC150	Deployable NLP
Resume skill extractor	Token classification	NER evaluation
Grounded FAQ bot	Embeddings/RAG/generator	Modern LLM system

19. Quick Revision

Pretrained transformer plus tokenizer. Attention: $\text{softmax}(\mathbf{QK}^T/\sqrt{d})\mathbf{V}$. Trap: mismatch/truncation. One-liner: attention lets each token select relevant context.

20. Final Cheat Sheet

Input/output: token IDs → logits/tokens. **Steps:** tokenize/forward/loss-or-generate/decode. **Keys:** max length/LR/decoding. **Pros:** transfer learning. **Cons:** quadratic context/hallucination. **Best:** language/multimodal tasks.

MLflow

1. Overview

MLflow tracks experiment parameters, metrics, artifacts, and model versions so ML results can be compared and reproduced.

2. Intuition

It is a laboratory notebook where every run stores settings, outcomes, and the resulting model.

3. Prerequisites

Python training code, artifacts/files, metrics, model serialization, and reproducibility.

4. Core Concepts

Concept	Meaning / why	Interview angle
Experiment/run	Named collection / one execution.	Why distinguish?
Params/metrics	Configuration / measured outcome.	Why different fields?
Artifact	Model, plot, report, data sample.	What must not be logged?
Tracking URI	Storage destination.	Local vs remote?
Registry	Versioned governed model entries.	Why aliases?

5. Algorithm / Working Process

Set tracking URI/experiment; start run; train; log params, metrics, artifacts; compare runs; register selected model; deploy a specific version/alias; monitor and retrain.

6. Mathematical Foundation

MLflow performs no optimization. It records objective $L(\theta; \lambda)$, hyperparameters λ , and metrics such as $accuracy = correct/N$.

7. Practical Implementation

```
import mlflow, mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2, random_state=42,
stratify=y)
mlflow.set_experiment("iris-classification")
with mlflow.start_run():
    C = 1.0; model = LogisticRegression(C=C, max_iter=300).fit(Xtr, ytr)
    mlflow.log_param("C", C); mlflow.log_metric("test_accuracy", model.score(Xte,
yte))
    mlflow.sklearn.log_model(model, name="model")
```

8. Code Explanation

The context manager safely ends the run. Parameters explain configuration, metrics support comparison, and the model flavor stores a loadable artifact.

9. Training / Evaluation

Log split strategy, seed, code/data version, feature signature, curves, validation metrics, and confusion matrix. Reserve final test data; never select repeatedly on it.

10. Complexity and Cost

Logging overhead is small, but checkpoints and images can dominate storage/network cost. Remote tracking needs backups, access control, and retention.

11. Common Use Cases

Hyperparameter selection, audit trail, controlled model promotion, scheduled retraining, and collaboration.

12. Common Mistakes

Logging PII/secrets, missing data/code version, comparing unequal splits, no schema, and treating a high metric as deployment approval.

13. Edge Cases / Limitations

It cannot ensure deterministic GPU behavior, data availability, fairness, or good monitoring by itself.

14. Variations

Local file tracking for learning; tracking server/object storage for teams; registry aliases for stable deployment references.

15. Related Topics

Pipelines package preprocessing; Docker freezes runtime; FastAPI serves artifacts; CI/CD controls promotion.

16. Interview Questions

1. Why MLflow? Reproducible comparable experiments.
2. Run vs experiment? Execution versus collection.
3. Param vs metric? Config versus measurement.
4. Artifact? Stored output file.
5. Why signature? Input/output validation.
6. Why log seed? Debug/reproduce.
7. Does MLflow train? No; it logs/orchestrates around training.
8. How choose model? Validation/business constraints then final test.
9. Why registry? Versioned governance/deployment.
10. What not log? Credentials and sensitive raw data.

17. Practice Tasks

Track three regularization values; log a confusion matrix; compare UI runs; register a model; reproduce a run from metadata.

18. Project Ideas

Project	Stack / dataset	Resume value
Churn experiment board	sklearn/MLflow; Telco	Reproducibility
Vision tracker	PyTorch/MLflow; CIFAR-10	Artifact management

Project	Stack / dataset	Resume value
Promotion demo	MLflow/FastAPI/Docker	End-to-end MLOps

19. Quick Revision

Experiment notebook for params, metrics, artifacts. Trap: no data version/PII. One-liner: MLflow gives lineage from training settings to model artifact.

20. Final Cheat Sheet

Input/output: run metadata/files → versioned model. **Steps:** start/log/compare/register. **Pros:** traceability. **Con:** storage/governance needed. **Best:** team MLOps.

FastAPI

1. Overview

FastAPI is a typed Python web framework for APIs. ML teams use it to expose prediction, embedding, retrieval, and health endpoints with OpenAPI documentation.

2. Intuition

It is a receptionist for a model: validate a request, invoke the model, and return a predictable JSON response.

3. Prerequisites

Python typing, JSON, HTTP methods/status codes, Pydantic, and basic concurrency.

4. Core Concepts

Concept	Meaning / why	Interview angle
Path operation	A route such as POST /predict.	POST vs GET?
Pydantic model	Typed input/output validation.	Why validate inference input?
Dependency	Reusable route concern.	Auth/model/database use?
Async	Cooperates during I/O waits.	Why not speed CPU work?
Lifespan	Loads shared resources once.	Why avoid per-request load?

5. Algorithm / Working Process

Server loads model; client sends JSON; schema validates it; route preprocesses and predicts; response is serialized with a status code. Heavy work needs appropriate workers/queues.

6. Mathematical Foundation

FastAPI adds no learning math. It exposes a function $x \mapsto f(x)$ and ensures x matches the expected input contract.

7. Practical Implementation

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

class Request(BaseModel): values: list[float] = Field(min_length=2, max_length=2)
class Response(BaseModel): label: int; score: float
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.weights = [0.8, -0.3] # replace with one-time model load
    yield
app = FastAPI(lifespan=lifespan)
@app.get("/health")
def health():
    return {"status": "ok"}
@app.post("/predict", response_model=Response)
def predict(request: Request):
    score = sum(x*w for x, w in zip(request.values, app.state.weights))
    if abs(score) > 1e6: raise HTTPException(422, "unsupported range")
    return Response(label=int(score > 0), score=score)
# uvicorn app:app --host 0.0.0.0 --port 8000
```

8. Code Explanation

Pydantic rejects malformed requests before inference. Lifespan holds one shared model. `response_model` documents and validates the output contract; health routes support deployment probes.

9. Training / Evaluation

The API does not train. Test routes and measure p50/p95 latency, throughput, error rate, input validation failures, and model quality/drift under realistic load.

10. Complexity and Cost

Framework overhead is small; model inference dominates. Synchronous CPU work blocks a worker; GPU memory limits replicas. Batching trades latency for throughput.

11. Common Use Cases

Online scoring, embeddings, RAG endpoints, internal ML services, and asynchronous job submission.

12. Common Mistakes

Model loading per request, blocking an async route with CPU work, absent auth/rate limits, unbounded payloads, stack-trace leakage, and no model version metadata.

13. Edge Cases / Limitations

Cold starts, malformed Unicode, traffic bursts, GPU OOM, and model/schema incompatibility need explicit handling and observability.

14. Variations

`async def` for I/O; background tasks for short post-response work; durable queues for long jobs; streaming for token generation.

15. Related Topics

Docker ships the service; MLflow supplies artifact versions; pipelines normalize raw requests.

16. Interview Questions

1. Why FastAPI? Typed validation, async support, docs.
2. GET vs POST? Retrieval versus body-submitted processing.
3. Why Pydantic? Reliable parsing/serialization.
4. Why lifespan? Load once and release cleanly.
5. Async benefit? I/O concurrency, not CPU acceleration.
6. How version models? Registry/model metadata and endpoint version.
7. What is p95? 95% of requests are faster than it.
8. How secure it? TLS, auth, validation, limits, secret manager.
9. Slow inference? Workers, batch, queue, timeout.
10. Why output schema? Stable client contract.

17. Practice Tasks

Serve a saved sklearn pipeline; add `/health` and metadata; validate ranges; write route tests; load-test it.

18. Project Ideas

Project	Stack / dataset	Resume value
Credit API	FastAPI/sklearn	Schema-safe serving
Semantic search	FastAPI/embeddings	Latency-aware NLP
Document classifier	FastAPI/queue	Async architecture

19. Quick Revision

Validated JSON → model → JSON. Trap: model load per request. One-liner: FastAPI creates documented typed ML endpoints.

20. Final Cheat Sheet

Input/output: JSON → JSON. **Steps:** schema, route, predict, serialize. **Keys:** workers/timeouts/model version. **Metrics:** latency/errors. **Pros:** fast to build. **Con:** security/ops required. **Best:** online inference.

Docker

1. Overview

Docker packages an application and dependencies into a portable image, then runs it as an isolated container. It makes ML jobs and services reproducible.

2. Intuition

An image is a sealed recipe; a container is a running copy of it.

3. Prerequisites

Linux process/filesystem basics, shell, ports, environment variables, Python dependencies, and image registries.

4. Core Concepts

Concept	Meaning / why	Interview angle
Image	Immutable layered runtime recipe.	Image vs container?
Container	Isolated running process.	Is it a VM?
Dockerfile	Build instructions.	Why layer order?
Volume	Persistent external storage.	Why not store model in writable layer?
Registry	Image store.	Why tag releases?

5. Algorithm / Working Process

Write Dockerfile; build and tag image; test; push to registry; host pulls and runs it with configured ports, secrets, volumes, and resource limits.

6. Mathematical Foundation

No learning math. Operationally, weights + activations + runtime overhead must fit the allocated CPU/GPU memory; image layers are reusable content-addressed filesystem layers.

7. Practical Implementation

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser
USER appuser
```

```
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.git
__pycache__/
.venv/
*.pt
*.pkl
```

8. Code Explanation

Copy dependencies before source to reuse cache. `.dockerignore` keeps caches, secrets, and heavy artifacts out of context. A non-root user lowers privilege.

9. Training / Evaluation

Build/run smoke tests; pin dependencies; scan image; check health endpoint and model checksum. Training containers should mount input/output artifacts and log image digest.

10. Complexity and Cost

Image size affects build/push/cold-start cost. CPU/RAM/GPU requests constrain runtime. GPU images need a compatible NVIDIA host driver/runtime.

11. Common Use Cases

FastAPI inference, reproducible training jobs, CI, batch inference, MLflow services, and Kubernetes workloads.

12. Common Mistakes

`latest` tags, secrets in image, huge context, root execution, mutable data in image, unpinned dependencies, and assuming CUDA image grants GPU access.

13. Edge Cases / Limitations

Containers share the host kernel, do not replace security hardening, and large models may produce unacceptable cold starts.

14. Variations

Multi-stage builds reduce size; Compose runs local stacks; Kubernetes orchestrates clusters.

15. Related Topics

FastAPI is often containerized; MLflow records image digests; CUDA-enabled containers run GPU workloads.

16. Interview Questions

1. Image vs container? Template versus running instance.
2. Container vs VM? Shared host kernel versus guest OS.
3. Why Docker? Reproducible runtime.
4. Why `.dockerignore`? Smaller, safer builds.
5. Why order layers? Cache reuse.
6. Why non-root? Smaller blast radius.
7. How persist data? Volumes/object store/database.
8. Why pin tags? `latest` changes.
9. How expose service? Port mapping/platform networking.
10. GPU support? Compatible host driver and GPU runtime.

17. Practice Tasks

Containerize FastAPI; add `.dockerignore`; prove caching; mount model volume; add a health smoke test.

18. Project Ideas

Project	Stack / dataset	Resume value
Container classifier	Docker/FastAPI/sklearn	Deployable artifact
Training job	Docker/PyTorch/MLflow	Reproducible environment
Local RAG stack	Compose/API/vector DB	Multi-service system

19. Quick Revision

Image is recipe, container is process. Trap: secrets, root, mutable tags. One-liner: Docker packages the userspace needed for the ML service.

20. Final Cheat Sheet

Input/output: Dockerfile → image → container. **Steps:** build/tag/run/publish. **Keys:** base/tag/port/volume. **Pros:** portable. **Con:** ops/security still needed. **Best:** repeatable services/jobs.

CUDA Programming Basics

1. Overview

CUDA is NVIDIA's platform for launching massively parallel GPU kernels. ML frameworks use it for matrix multiplication, attention, convolution, and reductions.

2. Intuition

Thousands of small workers each handle a small array region. Memory movement and coordination decide whether they are fast.

3. Prerequisites

C/C++ or Python GPU concepts, array indexing, parallelism, memory hierarchy, linear algebra, and race conditions.

4. Core Concepts

Concept	Meaning / why	Interview angle
Kernel	GPU-executed function.	Host vs device?
Thread/block/grid	Work unit/cooperating group/all blocks.	How index an element?
Warp	Usually 32 scheduled threads.	Why divergence hurts?
Memory	Registers/shared/global tiers.	Why coalesce?
Barrier	<code>__syncthreads()</code> within a block.	Why not across blocks?

5. Algorithm / Working Process

Allocate/copy device inputs; choose grid/block; launch kernel; each thread derives an index; load/compute/store; check error/synchronize; copy only if result is needed on host.

6. Mathematical Foundation

Vector add is $c_i = a_i + b_i$. Matrix multiplication is $C_{ij} = \sum_k A_{ik} B_{kj}$; tiled algorithms reuse blocks from shared memory to reduce global reads.

7. Practical Implementation

```
// vector_add.cu
#include <cuda_runtime.h>
__global__ void add(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
// int threads=256, blocks=(n+threads-1)/threads;
// add<<<blocks,threads>>>(a, b, c, n); cudaGetLastError();
cudaDeviceSynchronize();
```

8. Code Explanation

`__global__` runs on GPU and launches from host. The global index maps a thread to an element. Bounds checks protect rounded-up grids. Production code checks allocation/copy/launch errors too.

9. Training / Evaluation

CUDA is not trained: verify against NumPy/library results, warm up, synchronize before timing, profile memory bandwidth/occupancy, and benchmark realistic shapes.

10. Complexity and Cost

Vector add is $O(n)$, memory-bound. Matmul is $O(n^3)$, often compute-bound. PCIe transfers, noncoalesced loads, divergence, and limited VRAM are major costs.

11. Common Use Cases

DL primitives, image processing, simulation, Monte Carlo, scientific computing, and custom preprocessing.

12. Common Mistakes

No bounds checks, incorrect launch size, ignored errors, unsynchronized timing, races, excessive host-device transfer, and assuming all threads run at once.

13. Edge Cases / Limitations

Irregular branches serialize warp paths; tiny jobs cannot amortize launch cost; reductions need coordination; floating-point order varies; CUDA is NVIDIA-specific.

14. Variations

CUDA C++ is lowest level; Numba/CuPy are Python options; cuBLAS/cuDNN usually beat homemade standard kernels; Triton is a DL-oriented alternative.

15. Related Topics

PyTorch dispatches CUDA operations; Triton produces kernels; NCCL uses GPUs for distributed collectives.

16. Interview Questions

1. Kernel? GPU function launched by host.
2. Thread/block/grid? Work unit/group/collection.
3. Warp? Hardware scheduling group, commonly 32.
4. Divergence? Warp branches serialize.
5. Coalescing? Neighbor threads efficiently access neighbor addresses.
6. Shared vs global? Fast block-local scratchpad versus large device memory.
7. Why bounds check? Grid rounds up.
8. Why synchronize timing? Launch is asynchronous.
9. Occupancy? Active warps relative to capacity.
10. Why cuBLAS? Expert-tuned standard operations.

17. Practice Tasks

Implement vector scale; benchmark block sizes; write reduction; compare strided/coalesced loads; validate against NumPy.

18. Project Ideas

Project	Stack / dataset	Resume value
GPU filters	CUDA/OpenCV images	Thread/memory skills

Project	Stack / dataset	Resume value
Matmul study	CUDA/Nsight	Tiling analysis
PyTorch extension	CUDA/PyTorch	Framework-kernel bridge

19. Quick Revision

Many threads map to elements: $i = blockIdx.x \cdot blockDim.x + threadIdx.x$. Trap: transfer/timing errors. One-liner: CUDA exposes GPU threads and memory hierarchy.

20. Final Cheat Sheet

Input/output: device arrays → results. **Steps:** copy/launch/sync/validate. **Keys:** block size/layout. **Metrics:** bandwidth/occupancy/latency. **Pros:** performance. **Con:** complexity. **Best:** bottleneck kernels.

Triton Kernels

1. Overview

Triton is a Python-based GPU kernel language/compiler designed for high-performance deep-learning tensor programs.

2. Intuition

Each Triton program instance owns a tile of a tensor; you describe tiled loads, computation, and stores instead of manually managing CUDA threads.

3. Prerequisites

PyTorch devices/tensors, CUDA memory concepts, vectorized indexing, numerical verification, and profiling.

4. Core Concepts

Concept	Meaning / why	Interview angle
Program instance	One tile-owning work unit.	CUDA block analogy?
<code>program_id</code>	Identifies a tile.	How derive offsets?
Block size	Compile-time tile width.	Why tune it?
Mask	Safe tail load/store.	Why needed?
Fusion	One kernel does several operations.	Why faster?

5. Algorithm / Working Process

Select tile size; launch one program per tile; derive offsets; masked-load inputs; calculate vector/tile; masked-store output; benchmark configuration by shape/dtype/device.

6. Mathematical Foundation

For add, $y_i = x_i + b_i$. Fusion reduces memory traffic by avoiding intermediate global stores. Tiled matmul retains $C_{ij} = \sum_k A_{ik} B_{kj}$.

7. Practical Implementation

```
import torch, triton
import triton.language as tl
@triton.jit
def add_kernel(x, y, out, n: tl.constexpr, BLOCK: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n
    tl.store(out + offsets, tl.load(x + offsets, mask=mask) + tl.load(y + offsets,
    mask=mask), mask=mask)
def add(x, y):
    out = torch.empty_like(x); n = out.numel()
    add_kernel[(triton.cdiv(n, 256),)](x, y, out, n, BLOCK=256)
    return out
# assert torch.allclose(add(torch.ones(1000, device="cuda"), torch.ones(1000,
device="cuda")), torch.full((1000,), 2., device="cuda"))
```

8. Code Explanation

The decorator compiles the kernel. A program owns contiguous **BLOCK** offsets. The mask protects the final partial tile. Wrapper code sets the grid and allocates output.

9. Training / Evaluation

No training: test odd sizes and supported dtypes/strides against PyTorch using **allclose**; warm up, synchronize, repeat benchmarks, then profile end-to-end impact.

10. Complexity and Cost

Add stays $O(n)$, but fusion cuts launches and global memory traffic. Block choice controls register pressure, occupancy, and memory behavior; compilation costs first-run time.

11. Common Use Cases

Fused activation/normalization, attention pieces, optimizers, quantization, and LLM inference/training bottlenecks.

12. Common Mistakes

No tail mask, contiguous-only assumptions, cold benchmarking, unfair comparison to optimized libraries, bad numerical tolerance, and register overuse.

13. Edge Cases / Limitations

Standard library kernels can win; performance is hardware/version dependent; custom fusion can be harder to debug.

14. Variations

Autotuning selects blocks/warps; 2D programs implement tiled matmul; chains fuse pointwise operations. Valuable for LLM systems roles.

15. Related Topics

CUDA explains hardware; PyTorch runs kernels; `torch.compile` may generate fusion automatically.

16. Interview Questions

1. Triton? GPU tensor kernel DSL/compiler.
2. Why over CUDA? Higher-level tiled abstraction.
3. Program ID? Tile identifier.
4. Why mask? Safe nonmultiple tails.
5. Fusion benefit? Fewer launches/intermediates.
6. Why tune blocks? Balance registers, occupancy, overhead.
7. Verify? Compare trusted reference over cases.
8. Benchmark? Warm/sync/repeat realistic shapes.
9. Always faster? No.
10. Autotuning? Empirical config selection.

17. Practice Tasks

Implement ReLU; fuse add+ReLU; test tail sizes; compare blocks; build tiled matmul after reading a reference.

18. Project Ideas

Project	Stack / dataset	Resume value
Fused MLP benchmark	Triton/PyTorch	Fusion evidence
LayerNorm study	Triton/profiler	Reduction tuning
LLM micro-optimizer	Triton/PyTorch	Systems specialization

19. Quick Revision

Programs own tiles; masks handle tails; fusion reduces traffic. Trap: unverified speed claims. One-liner: Triton writes fused tiled GPU tensor code in Python syntax.

20. Final Cheat Sheet

Input/output: GPU pointers → tensor tiles. **Steps:** grid/offset/mask/load/compute/store. **Keys:** blocks/warps. **Metrics:** correctness/latency. **Pros:** productive fusion. **Con:** hardware-sensitive. **Best:** profiled DL bottlenecks.

Distributed Training Internals

1. Overview

Distributed training uses several GPUs or machines to increase training throughput or fit models/data beyond one device. It powers LLM, vision, and recommender training.

2. Intuition

Each worker has the same model, receives a different mini-batch, calculates what it learned, averages that learning signal with peers, and updates identically.

3. Prerequisites

Backpropagation, PyTorch loops, GPU memory, networking basics, batching, collectives, and floating-point behavior.

4. Core Concepts

Concept	Meaning / why	Interview angle
World size/rank	Process count / one process ID.	Why one process per GPU?
Data parallelism	Replicate model, shard batch.	What is all-reduce?
All-reduce	Aggregate gradients to every rank.	Why must states align?
DDP	PyTorch gradient-sync wrapper.	Why faster than DataParallel?
Sharding	Split model/optimizer state.	FSDP vs DDP?
Checkpoint	Training state for resume.	What must be saved?

5. Algorithm / Working Process

Initialize process group; assign each rank one GPU; shard data with `DistributedSampler`; forward locally; backward produces gradient buckets; all-reduce averages them, often overlapping backward; identical optimizer steps follow. DDP replicates states; FSDP/ZERO shard them.

6. Mathematical Foundation

Worker r computes $g_r = \nabla L_r(\theta)$. Synchronous data parallel training uses $g = W^{-1} \sum_{r=1}^W g_r$, matching a global-batch gradient under normal assumptions. Communication is roughly parameter-size scale per step.

7. Practical Implementation

```
# Run: torchrun --nproc_per_node=2 train.py
import os, torch
import torch.distributed as dist
from torch import nn
```

```
from torch.nn.parallel import DistributedDataParallel as DDP
dist.init_process_group("nccl")
rank = int(os.environ["LOCAL_RANK"]); torch.cuda.set_device(rank)
model = DDP(nn.Linear(10, 2).cuda(rank), device_ids=[rank])
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
x = torch.randn(8, 10, device=rank); y = torch.randint(2, (8,), device=rank)
loss = nn.CrossEntropyLoss()(model(x), y)
opt.zero_grad(); loss.backward(); opt.step()
dist.destroy_process_group()
```

8. Code Explanation

`torchrun` creates ranks/environment variables. Each process owns one GPU. DDP inserts gradient all-reduce during backward, so each rank performs the same update. Real data loading needs `DistributedSampler` and `sampler.set_epoch(epoch)`.

9. Training / Evaluation

Global batch is local batch $\times$ world size; adjust learning rate carefully, often with warmup. Aggregate metrics across ranks. Save model, optimizer, scheduler, AMP scaler, RNG state, and data position. Profile input stalls and communication.

10. Complexity and Cost

Compute per rank falls with more GPUs, but communication, stragglers, and input throughput limit scaling. DDP duplicates parameters/optimizer state; FSDP/ZeRO lower memory but add communication and complexity. Interconnect quality matters.

11. Common Use Cases

Multi-GPU vision, LLM pretraining/fine-tuning, large recommendation models, and high-throughput embedding training.

12. Common Mistakes

Duplicate samples across ranks, missing `set_epoch`, all ranks logging/checkpointing, unaggregated metrics, rank-divergent control flow, missing optimizer state, and assuming linear speedup.

13. Edge Cases / Limitations

Uneven batches, node failures, network timeouts, collective deadlocks, non-determinism, and model state too large for replicated memory require robust operations.

14. Variations

DDP: standard data parallelism. FSDP/ZeRO: shard states. Tensor parallelism: split layer math. Pipeline parallelism: split model layers. Sequence parallelism: long-context work. DDP is placement-critical; other forms are advanced.

15. Related Topics

CUDA/NCCL execute collectives; PyTorch DDP provides API; MLflow tracks multi-GPU runs; Docker standardizes nodes; Triton optimizes local kernels.

16. Interview Questions

1. Data parallelism? Replicate model, split data, sync gradients.
2. All-reduce? Aggregate then distribute result to all ranks.
3. One process/GPU? Clear ownership and efficient communication.
4. Global batch? Local batch times world size.
5. Why sampler? Distinct data shards.
6. Why `set_epoch`? New deterministic shuffle each epoch.
7. DDP vs DataParallel? Multi-process efficient communication versus single-process bottleneck.
8. Scaling limit? Communication, stragglers, memory, input pipeline.
9. FSDP benefit? State sharding fits larger models.
10. Deadlock? Some ranks wait for a collective others never reach.

17. Practice Tasks

Convert a single-GPU model to DDP; verify unique sample IDs per rank; aggregate validation metrics; resume checkpoint; measure 1-vs-2 GPU throughput.

18. Project Ideas

Project	Stack / dataset	Resume value
Multi-GPU image trainer	PyTorch DDP/ImageNet subset	Correct data sharding
FSDP LLM fine-tune	PyTorch/FSDP/LoRA	Memory-aware LLM training
Scaling dashboard	PyTorch/MLflow/profiler	Systems measurement

19. Quick Revision

Replicate model, shard data, average gradients: $g = W^{-1} \sum g_r$. Trap: duplicated samples/rank-unsafe checkpoints. One-liner: DDP overlaps gradient all-reduce with backpropagation.

20. Final Cheat Sheet

Input/output: data shards → synchronized update. **Steps:** init/shard/forward-backward/all-reduce/step. **Keys:** world size/local batch/LR/backend. **Metrics:** throughput/scaling efficiency/loss. **Pros:** scale. **Con:** communication/failure complexity. **Best:** multi-GPU DL.